TEST_COVERAGE_REPORT

HelixCluster — Test Coverage Report

Field	Value
Revision	1
Generated	2026-06-03
Author	AI agent (claude-opus-4-8)
Authority	CLAUDE-1 (anti-bluff), CONST-050 (100% test-type coverage), §11.4
Host of measurement	macOS arm64 (Darwin 24.5.0), Go 1.26.4 toolchain
Method	`go test ./... -coverprofile -covermode=atomic` per module + `go tool cover -func`; test-type inventory by build-tag / function-prefix / directory

> **Honesty note (CLAUDE-1).** Go measures coverage **per package (statement coverage)**, not per *test type* — there is no native "% covered by integration tests" metric. This report therefore states (a) **real measured statement coverage per module**, and (b) a **per-test-type inventory** (counts + what each type verifies + execution status). Numbers below are the actual measured output of the commands in the Method row, not estimates.

---

1. Measured statement coverage (real)

Module	Statement coverage	Packages in profile	Notes
`github.com/HelixDevelopment/helix_cluster` (main)	**82.4%**	255	Unit-level sweep (`go test ./...`, no integration tag).
`digital.vasic.security` (security submodule)	**87.8%**	—	Crypto/e2ee/guardrails/pii/content/policy/scanner.
`github.com/HelixDevelopment/helix_cluster/api/v1`	**14.7%**	1	Generated protobuf stubs — low coverage is expected for

Module	Statement coverage	Packages in profile	Notes
			codegen; the round-trip tests (`roundtrip_test.go`) exercise the wire (marshal/unmarshal) path, not every generated getter.

Coverage profiles are captured under `qa-results/coverage/` (gitignored, local evidence). Aggregate caveat: the main sweep descends into `web/node_modules/flatted/golang` (a vendored JS-tooling directory containing an embedded Go test that fails); it is **not first-party code**. Recommended follow-up: exclude `web/node_modules` from the test sweep (see Known gaps §4).

The local coverage gate machinery exists in `pkg/covgate` (`MeetsThreshold`/`Shortfalls`, `scripts/test.sh COVERAGE_THRESHOLD=80`) but is not enforced by an automated gate (CI is disabled by constitutional mandate; see §4).

---

2. Per-test-type inventory (CONST-050 supported types)

Counts are first-party `*_test.go` (excluding `HelixConstitution/` and vendored trees) and challenge scripts. "Execution status" reflects what was actually run/verified this development cycle on the measurement host.

Test type	Inventory	How identified	Execution status
Unit	~609 `*_test.go` (pkg 443 · internal 101 · cmd 34 · security 30 · api/v1 1)	default test files	RUN — feeds the §1 statement coverage (82.4% / 87.8%).
Integration (real services)	104 build-tagged files (`//go:build integration`)	build tag	RUN against REAL services this cycle via rootless podman + `digital.vasic.containers/pkg/brokertest`: `pkg/etcd`, `pkg/lock`, `pkg/leader`, `pkg/discovery`, `pkg/hxcregistry`, `internal/node`, `cmd/helix-agent` (all green under `-race`); `pkg/events` Avro-over-live-NATS. Remaining integration files run when their service is provided.
End-to-end (E2E)	214 files (name/`//go:build` markers)	name + tag	Component-level E2E run on host; full distributed E2E (`workload→client→E2EE→network→miner→response`) is **infra-gated** — tracked **HXC-1491** (needs a deployed multi-node cluster + real GPU providers).
Stress			

Test type	Inventory	How identified	Execution status
	part of 117 stress/chaos files	name match `stress`	RUN where host-closeable (e.g. e2ee concurrent framed-transport `-race`, Avro concurrent round-trip, SELinux relabel stress).
Chaos	part of 117 stress/chaos files	name match `chaos`	RUN where host-closeable (fault-injection in e2ee/ SELinux paths); cluster-wide chaos is infra-gated.
Benchmark / performance	211 `func Benchmark*`	function prefix	Present; run on demand via `make benchmark`.
Fuzz	20 `func Fuzz*`	function prefix	Present (e.g. `pkg/covgate/parse_fuzz_test.go`, `pkg/tracing/w3c_fuzz_test.go`); run on demand.
Race	(mode, not a count)	`go test -race`	RUN — `make test` + `scripts/test.sh unit` use `-race`; this cycle's gated packages were all `-race`-clean.
VM / cross-arch	1 build-tagged (`//go:build vm`)	build tag	`make vm-test` (`./tests/vm_nodes/...`); requires VM/QEMU substrate (infra-gated).
Security / vulnerability	govulncheck + SBOM	tooling	RUN — `govulncheck ./...` over main/api-v1/ security = **0 reachable advisories** (after HXC-1631/1632); `make sbom` (CycloneDX, HXC-1635).
Challenges (HelixQA)	55 `challenges/` entries · 52 challenge scripts · `helixqa/` module	dirs	Framework present; a full autonomous HelixQA session with per-feature sink-side evidence was **not executed** this cycle — tracked (PRR item 21).

---

3. What is verified REAL-STACK vs component-only

Verified against real services / real evidence this cycle (sink-side, anti-bluff): - etcd coordination layer (`etcd/lock/leader/discovery/hxcregistry/node`) — real etcd, `-race`. - Event bus — Avro single-object payloads over a **real NATS+JetStream broker** (`brokertest.StartNATS`), on-wire `0xC3 0x01` marker asserted. - DB migrations — `make migrate-up/down` against a **real postgres** (15 migrations → 22 tables → rollback). - Crypto — ChaCha20-Poly1305 **RFC-8439 byte-exact KAT**; e2ee framed transport + per-request keypair isolation under `-race`. - WireGuard macOS — **real gVisor-netstack userspace tunnel** moving real TCP traffic (no root). - GPU probe (darwin) — real `system_profiler` oracle match.

Component-only (NOT yet validated end-to-end on real infrastructure): - The full multi-node request pipeline (HXC-1491) and the multi-cell usability testbed (HXC-1384) — both **P0 Queued, infra-gated**. “All green” at the unit/integration level does **not** assert production-grade end-to-end operation across the full T1–T8 tier matrix.

---

4. Known coverage gaps & caveats

1. **No continuous gate.** CI/CD is constitutionally forbidden (local gates only), so coverage, `-race`, `govulncheck`, and the `pkg/covgate` 80% threshold are **not auto-enforced** on push; they run via `make test` / `scripts/test.sh` / `make deps-update` manually. Regressions are not caught automatically.
2. **`web/node_modules` in the test sweep.** `go test ./...` descends into a vendored JS-tooling directory whose embedded Go test fails; exclude it from the sweep (it is not first-party code).
3. **api/v1 low coverage is by design** (generated stubs); the meaningful guarantee is the round-trip wire test, not generated-getter coverage.
4. **HelixQA challenges not executed** this cycle for per-feature sink-side evidence (PRR item 21).
5. **Infra-gated test types** (full E2E, cluster chaos, VM matrix) cannot be exercised on the single macOS host; they require a deployed cluster / VM substrate / real hardware.

---

5. Reproduce

```
# per-module statement coverage
go test ./... -count=1 -coverprofile=cover.out -covermode=atomic
        && go tool cover -func=cover.out | tail -1
(cd security && go test ./... -count=1 -cover)
(cd api/v1  && go test ./... -count=1 -cover)
# real-service integration (rootless podman must be running)
go test -tags=integration -race ./pkg/etcd/... ./pkg/lock/... ./
        pkg/leader/... ./pkg/discovery/...
# security + supply chain
govulncheck ./...        # 0 reachable advisories
make sbom                # CycloneDX SBOMs
```